

A
MINI PROJECT REPORT
ON

Smart Voice Search for Contact Management WhatsApp (Clone)

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN
COMPUTER SCIENCE AND ENGINEERING

Submitted by

M.UPENDRA CHARI (22R91A05G1)

Under the Guidance

Of

Mr. P. V. RAMA GOPAL RAO

(Assistant Professor)

Department of CSE



Department of Computer Science and Engineering

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(An Autonomous Institution)

Medbowli, Meerpet, Saroornagar, Hyderabad – 500097

(Affiliated to JNTUH, Approved by AICTE, Accredited by NBA & NAAC)

(2022-2026)



TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(Sponsored by TKR Educational Society)

Approved by AICTE, Affiliated by JNTUH, Accredited by NBA & NAAC)

Medbowli, Meerpet, Saroornagar, Hyderabad – 500 097.

Phone: 040-24092838 Fax: +91-040-24092555

E-mail: tkrec@rediffmail.com Website: www.tkrec.ac.in

Department of Computer Science & Engineering

College code: R9

CERTIFICATE

This is to certify that the Mini Project report on “**SMART VOICE SEARCH FOR CONTACT MANAGEMENT WHATSAPP (CLONE)**” is a bonafide work carried out by **M.UPENDRA CHARI (22R91A05G1)** in partial fulfillment for the requirement of the award of B.Tech degree in Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Hyderabad, affiliated to Jawaharlal Nehru Technological University, Hyderabad under my guidance and supervision.

The result of investigation enclosed in this report have been verified and found satisfactory. The results embodied in the project work have not been submitted to any other University for the award of any degree.

INTERNAL GUIDE

Mr. P. V. RAMA GOPAL RAO
Assistant Professor

.....

HEAD OF DEPARTMENT

Dr. CH. V. Phani Krishna
Professor

.....

EXTERNAL EXAMINER

.....

PRINCIPAL

Dr. K. Venkata Murali Mohan
Professor

.....



TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(Sponsored by TKR Educational Society)

Approved by AICTE, Affiliated by JNTUH, Accredited by NBA & NAAC)

Medbowli, Meerpet, Saroornagar, Hyderabad – 500 097.

Phone: 040-24092838 Fax: +91-040-24092555

E-mail: tkrec@rediffmail.com Website: www.tkrec.ac.in

Department of Computer Science & Engineering

College code: R9

DECLARATION

I hereby declare that the Mini Project report entitled “**Smart Voice Search for Contact Management WhatsApp(Clone)**” is done under the guidance of **Mr.P. V. RAMA GOPAL RAO**, Assistant Professor, Department of Computer Science and Engineering, Teegala Krishna Reddy Engineering College, is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from **Jawaharlal Nehru Technological University, Hyderabad**.

This is a record of bonafide work carried out by us in **Teegala Krishna Reddy Engineering College** and the results embodied in this project have not been reproduced or copied from any source.

Submitted by

M.UPENDRA CHARI (22R91A05G1)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragement and guidance have crowned our efforts with success.

I extend my deep sense of gratitude to **Dr.K.Venkata Murali Mohan, Principal** Teegala Krishna Reddy Engineering College, Meerpet, for permitting me to undertake this project.

I am indebted to **Dr.CH.V. Phani Krishna, Professor & Head of the Department**, Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Meerpet for his support and guidance throughout our project.

I am indebted to our guide **Mr.P.V. RAMA GOPAL RAO**, Assistant Professor Computer Science Engineering, Teegala Krishna Reddy Engineering College, Meerpet for his/her support and guidance throughout our project.

I am indebted to the project coordinators **P Swetha**, Assistant professor, Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Meerpet for her support and guidance throughout our project.

Finally, we express thanks to one and all that have helped me in successfully completing this Mini project. Further I would like to thank my family and friends for their moral support and encouragement.

By

M.UPENDRA CHARI (22R91A05G1)

ABSTRACT

The project titled "Smart Voice Search for Contact Management" is a dynamic web-based application that replicates a WhatsApp-like interface while introducing innovative voice-enabled contact search capabilities. Designed with user convenience and accessibility in mind, it allows users to efficiently search for contacts through either traditional text input or voice commands, leveraging the Web Speech API for real-time speech recognition. Upon selecting a contact, users are given options to initiate a phone call or send a message, simulating core messaging app functionalities. The interface also features a comprehensive "More Options" menu, providing access to group creation, device linking, payment management, and settings—extending the application's usability beyond basic communication. Unlike conventional systems that rely heavily on manual search, this system significantly enhances user experience through a faster, hands-free interaction model, making it ideal for users with mobility challenges or in multitasking environments.

LIST OF FIGURES

F.NO	NAME	PAGE NO
4.2.1	Class Diagram	8
4.2.2	Use Case Diagram	9
4.2.3	Sequence Diagram	10
4.2.4	Activity Diagram	11

CONTENTS

S. NO	TITLE	PAGE NO
1	Introduction	1
2	Literature Survey	2
3	System Analysis	3
	3.1 Existing System	4
	3.2 Proposed System	5
	3.3 System Requirements	5
	3.3.1 Hardware Requirements	6
	3.3.2 Software Requirements	
4	System Design	7
	4.1 System Architecture	7
	4.1.1 Frontend (Client Side)	7
	4.1.2 Backend (Server Side)	7
	4.1.3 Database (Mongo DB)	8
	4.2 UML Diagrams	9-10
	4.2.1 Class Diagrams	11-12
	4.2.2 Use-Case Diagrams	13
	4.2.3 Sequence Diagrams	14
	4.2.4 Activity Diagrams	
5	Implementation	15
	5.1 Technologies	15
	5.2 Tools	15
	5.2.1 Visual Studio Code	15
	5.3 Modules	15-16
6	Coding	17-23
7	Output Screens	24-25
8	Conclusion	26-27
9	Reference	28

1.INTRODUCTION

The “Smart Voice Search for Contact Management” project is an innovative web-based application developed to streamline and enhance the user experience in managing and interacting with contact lists. Inspired by the user interface and features of WhatsApp, this system allows users to search for contacts using either traditional text input or voice commands, making it especially useful for hands-free operation and accessibility. At its core, the application integrates the Web Speech API, a browser-based technology that captures and converts spoken language into text, enabling quick and efficient contact filtering.

When a contact is selected, users are presented with intuitive options to initiate a call or send a message, thereby replicating essential messaging app functionalities. In addition to these basic actions, the system incorporates a versatile More Options Menu, offering users access to advanced features such as group creation, device linking, payment management, and settings control, making it a robust and interactive contact management platform.

The project aims to overcome the limitations of conventional contact systems that rely heavily on manual text entry and lack accessibility features. With an increasing emphasis on voice-assisted technologies in modern applications, this project showcases how voice recognition can be effectively leveraged to improve usability, reduce interaction time, and accommodate users with physical limitations. By combining a user-friendly interface with powerful backend support through Node.js, Express.js, and MongoDB, this project not only provides a practical solution for contact management but also serves as a demonstration of how emerging web technologies can be applied in real-world scenarios. Ultimately, this application highlights the potential of integrating voice-enabled AI tools into everyday digital environments, paving the way for smarter and more responsive user experiences.

2.LITERATURE SURVEY

In recent years, the demand for smarter and more accessible user interfaces has driven a surge in the adoption of voice recognition technologies across web and mobile applications. Researchers and developers have increasingly explored the integration of speech-based interaction to enhance usability, particularly in environments where traditional input methods are inconvenient or inaccessible. The Web Speech API, developed under the W3C Community Group and supported by browsers like Chrome, enables developers to capture voice input and convert it to text using the `webkitSpeechRecognition` interface.

This has laid the foundation for building intelligent, real-time voice-controlled features within web applications. Numerous academic studies have highlighted the benefits of using voice commands in systems ranging from virtual assistants (e.g., Google Assistant, Amazon Alexa) to healthcare interfaces and smart home automation, showing measurable improvements in user engagement and operational efficiency.

Additionally, research on contact management systems has pointed out the limitations of conventional search techniques, which rely on manual text input and linear data filtering. Traditional contact apps often lack dynamic filtering, voice accessibility, and personalized interactions, which results in a sub-optimal experience—especially for users with disabilities or in fast-paced scenarios. Recent literature emphasizes the importance of natural language processing (NLP), speech-to-text algorithms, and responsive UI design in addressing these gaps.

Developers have been leveraging full-stack technologies—Node.js, Express.js, MongoDB, and JavaScript—to create scalable, modular, and real-time data-driven platforms. Studies also indicate that integrating NoSQL databases like MongoDB improves flexibility in managing unstructured contact data, while tools like Mongoose streamline schema design and database operations.

3.SYSTEM ANALYSIS

The system analysis for the project titled “**Smart Voice Search for Contact Management**” (**WhatsApp Clone**) focuses on developing an interactive, voice-enabled web application that enhances the traditional contact search process. Conventional systems rely solely on manual text input, which can be tedious, time-consuming, and less inclusive for users with disabilities.

The proposed system integrates the **Web Speech API**, allowing users to search for contacts using voice commands, making the process faster, more efficient, and accessible. The user interface is inspired by WhatsApp, offering familiar navigation with options to call or message selected contacts. Key modules include the user interface, contact management, real-time search filtering, voice recognition, contact actions, and navigation controls.

The system uses technologies such as **HTML, CSS, JavaScript, Node.js, and MongoDB**, ensuring responsiveness, scalability, and maintainability. Additional features like group creation, device linking, and settings personalization enhance functionality. Voice input is processed through `webkitSpeechRecognition`, converting speech to text and triggering real-time search. This application is particularly useful for multitasking environments and supports users with physical or visual limitations. The architecture supports modular development and future enhancements, such as language translation or AI-powered suggestions. Overall, this smart contact manager represents a modern, user-friendly, and inclusive approach to web-based communication tools.

Key system modules include a user interface with voice input, a dynamic contact management system, real-time search filtering, and contact action features. The voice recognition module captures spoken input, converts it into text, and triggers contact filtering automatically. Supporting features such as group creation, linked device management, and settings access enrich the user experience.

3.1 Existing system

- Most contact management applications depend solely on manual text-based search.
 - Users must type the contact name to locate entries, which can be time-consuming.
 - There is no support for voice input or voice search, limiting accessibility.
 - The system lacks real-time filtering and intelligent search capabilities.
 - User experience is basic, with no integration of quick actions like direct call or message options.
 - Hands-free operation is not possible, making it unsuitable for multitasking environments.
 - Limited in accessibility for users with physical impairments or special needs.
-
- Does not offer dynamic interfaces or modern UI elements similar to popular messaging platforms.
 - The absence of smart features reduces user engagement and interaction speed.
 - Overall, the current system is outdated and inefficient for modern user expectations.

DISADVANTAGES

- Lack of Voice Search Integration
- Manual Input Dependency
- Limited Accessibility and User Experience

3.2 Proposed System

- Develop a web-based contact management application.
- Integrate voice search functionality using the Web Speech API.
- Allow users to search for contacts via voice commands or typed input.
- Provide a WhatsApp-like interface for better user familiarity and ease of use.
- Improve accessibility and efficiency in searching and managing contacts.

Advantages

- Voice Search Feature
- Quick Contact Actions
- Back Navigation Support
- Interactive User Interface

3.3 System Requirements

3.3.1 Hardware Requirements

COMPONENTS	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB or higher
Storage	At least 500 MB of free space
Microphone	Built-in or external (for voice input)

3.3.2 Software Requirements

Software

Description

Operating System

Windows 7/8/10/11, macOS, or Linux

4.SYSTEM DESIGN

The system is a web-based contact management application designed to provide a modern, user-friendly experience for searching and managing contacts. Inspired by the interface and usability of popular messaging platforms like WhatsApp, it offers both voice-based and text-based search functionality, enhancing convenience and accessibility for users.

4.1 SYSTEM ARCHITECTURE

4.1.1. Frontend (Client Side):

- Displays UI with contact list, search bar, and voice input button.
- Uses Web Speech API (webkitSpeechRecognition) to capture and convert voice to text.
- Sends search queries to backend via HTTP (AJAX/Fetch).
- Shows filtered contact results and provides “Call” or “Message” options.

4.1.2. Backend (Server Side):

- Built using Node.js and Express.js.
- Exposes REST API routes like:
- GET /contacts → fetch all contacts
- GET /contacts,search=query → fetch contacts matching search
- Applies filtering logic for search (case-insensitive match).
- Handles request/response communication between frontend and database.

4.1.3. Database (MongoDB):

- The Smart Voice Search for Contact Management application uses MongoDB as its backend database.
- MongoDB is a widely-used NoSQL database that stores data in JSON-like documents (BSON), making it highly suitable for flexible and scalable data models required in modern web applications

4.2 UML Diagram

UML (Unified Modeling Language) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of a software system, as well as business modeling and other non-software systems. The UML represents a common ground for modeling software by providing a graphical notation and a set of underlying concepts that represent common elements of software systems. The UML can be used to model a wide range of software systems, from small single-user applications to large, complex enterprise systems.

It can be used to model both the static and dynamic aspects of a system, including its structure, behavior, and interactions with other systems. UML diagrams are used to create visual representations of a system's design. They can be used to show the different parts of a system, how they interact with each other, and how they work together to achieve a common goal. Class diagrams show the classes in a system and how they are related to each other. Object diagrams show instances of classes and how they are related to each other. Sequence diagrams show the interactions between objects in a system over time. Collaboration diagrams show the interactions between objects in a system at a specific point in time. State machine diagrams show the different states that an object can be in and how it can transition between those states. Activity diagrams show the workflow of a process, such as a business process.

Deployment diagrams show the physical deployment of a system's components, such as hardware and software. SRSs describe the functional and non-functional requirements of a software system. UML diagrams can be used to illustrate the requirements in an SRS. SDDs describe the design of a software system. UML diagrams can be used to illustrate the design in an SDD. UI prototypes are early mockups of a software system's UI. UML diagrams can be used to create wireframes and storyboards for UI prototypes. Test cases describe the steps that should be taken to test a software system. UML diagrams can be used to illustrate the steps in a test case. Deployment plans: Deployment plans describe how a software system will be deployed to production.

UML diagrams can be used to illustrate the deployment process. Unified Modeling Language (UML) is a standardized language for specifying, visualizing, constructing, and documenting the artifacts of a software system, as well as business modeling and other non-software systems. UML diagrams are a graphical representation of a system, including its components, structure, and behavior. They are used to communicate and document the design of a system to all stakeholders involved in its development and maintenance.

4.2.1. Class Diagram

The class diagram presents the structural blueprint of the system by outlining its key classes, their attributes, and associated methods. Central to the application are five core classes: Contact, UserInterface, VoiceRecognition, SearchEngine, and ActionHandler. The Contact class encapsulates essential contact details such as name, phone number, photo, and email, along with methods like `getDetails()` and `isFavorite()` to manage individual contact functionalities.

The UserInterface class governs the application's visual layout and themes, offering methods such as `displayContacts()` and `showSearch()` to facilitate user interaction. Voice input functionality is handled by the VoiceRecognition class, which includes attributes like language and transcript, and provides the methods `startListening()` and `stopListening()` to capture and process spoken input.

The SearchEngine class is responsible for executing both text and voice-based searches through methods like `searchByText()` and `searchByVoice()`, operating on a query attribute that holds the user's search input. Lastly, the ActionHandler class manages interactions with selected contacts, enabling users to initiate actions such as `call()` or `message()`. Overall, this class diagram emphasizes the organized distribution of data and responsibilities, promoting a modular and maintainable system design.

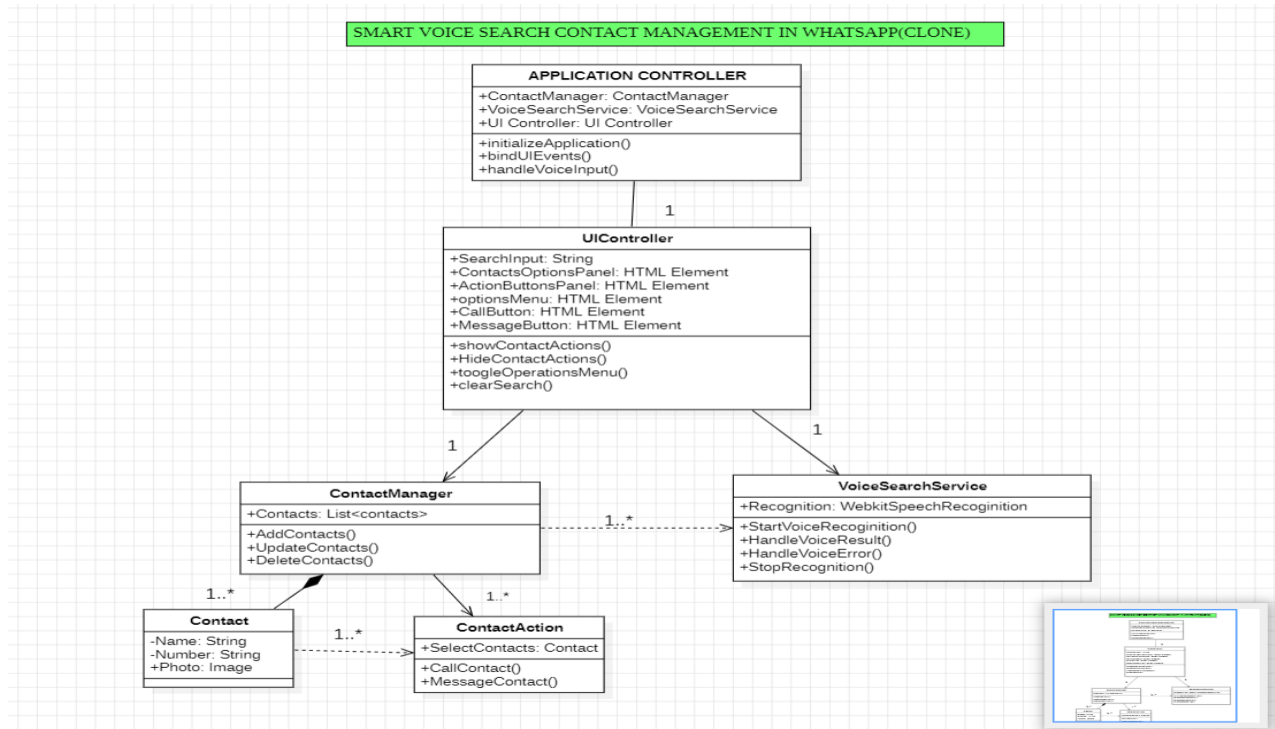


Fig.4.2.1: Class Diagram

4.2.2 Use Case Diagram

The use case diagram for the project "Smart Voice Search for Contact Management" represents the functional interactions between the user and the system. In this web-based application, the primary actor is the user, who can perform various actions such as searching for contacts using either text or voice input through the Web Speech API.

Once a contact is found, the user can initiate a call or send a message. Additional features include group creation, device linking, managing payments, and accessing settings. The diagram illustrates how the user interacts with different modules, enhancing contact accessibility and overall user experience through voice-enabled navigation

SMART VOICE SEARCH FOR CONTACT MANAGEMENT IN WHATSAPP(CLONE)

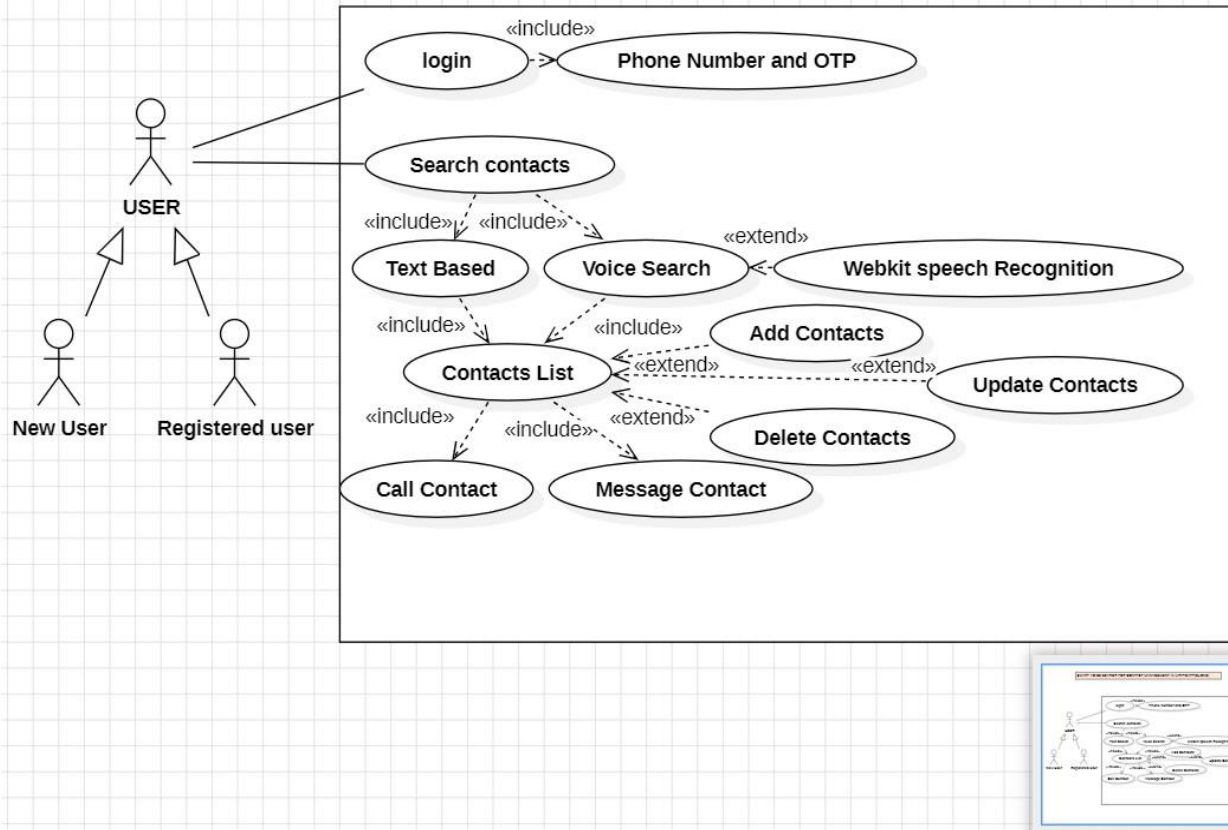


Fig 4.2.2 Use Case Diagram

4.2.3 SEQUENCE Diagram

The sequence diagram for the "Smart Voice Search for Contact Management" project illustrates the dynamic interactions between the user and the various system modules in a time-sequenced manner. The process begins when the user launches the web application, prompting the user interface to load. The user then chooses to search for a contact using either text input or voice command. If voice input is selected, the Voice Recognition Module activates the Web Speech API, captures the spoken input, and converts it into text. This text is then sent to the Search Functionality Module, which processes the input and filters the contact list accordingly. The updated contact list is displayed to the user in real-time. The user then selects a specific contact, triggering the Contact Action Module to present interaction options such as "Call" or "Message." Once an option is selected, the system executes the corresponding function and provides feedback. The sequence concludes with the user returning to the main contact interface.

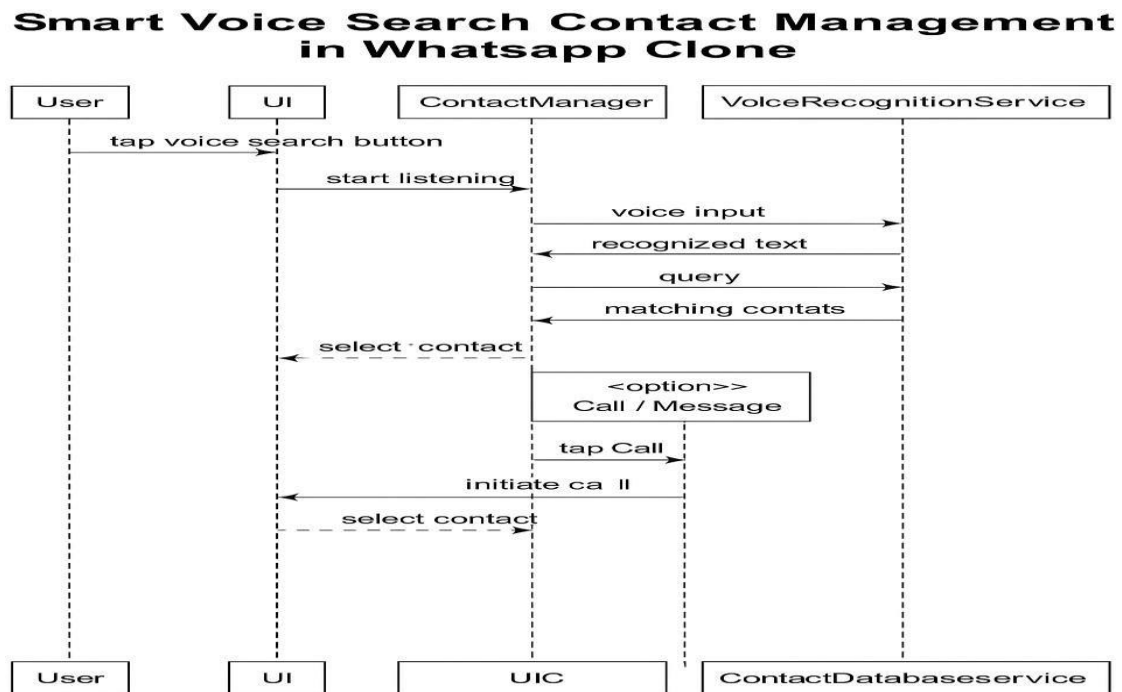


Fig 4.2.3 Sequence Diagram

4.2.4 Activity Diagram

The activity diagram for the "Smart Voice Search for Contact Management" project illustrates the sequence of user and system activities from the initiation of the application to the completion of contact-related actions. The process starts with the user launching the web application, which loads the WhatsApp-like interface. The user then decides whether to use text input or voice input to search for a contact. If voice input is chosen, the system activates the Web Speech API to capture and convert spoken words into text. This converted input, or manually typed text, is used by the search module to filter and display matching contacts in real time. The user can then select a contact from the list, upon which options such as "Call" or "Message" become available. Once an action is selected, the system processes the request and returns to the main contact list. Additional navigation includes options like group creation, managing linked devices, payment settings, and accessing app settings

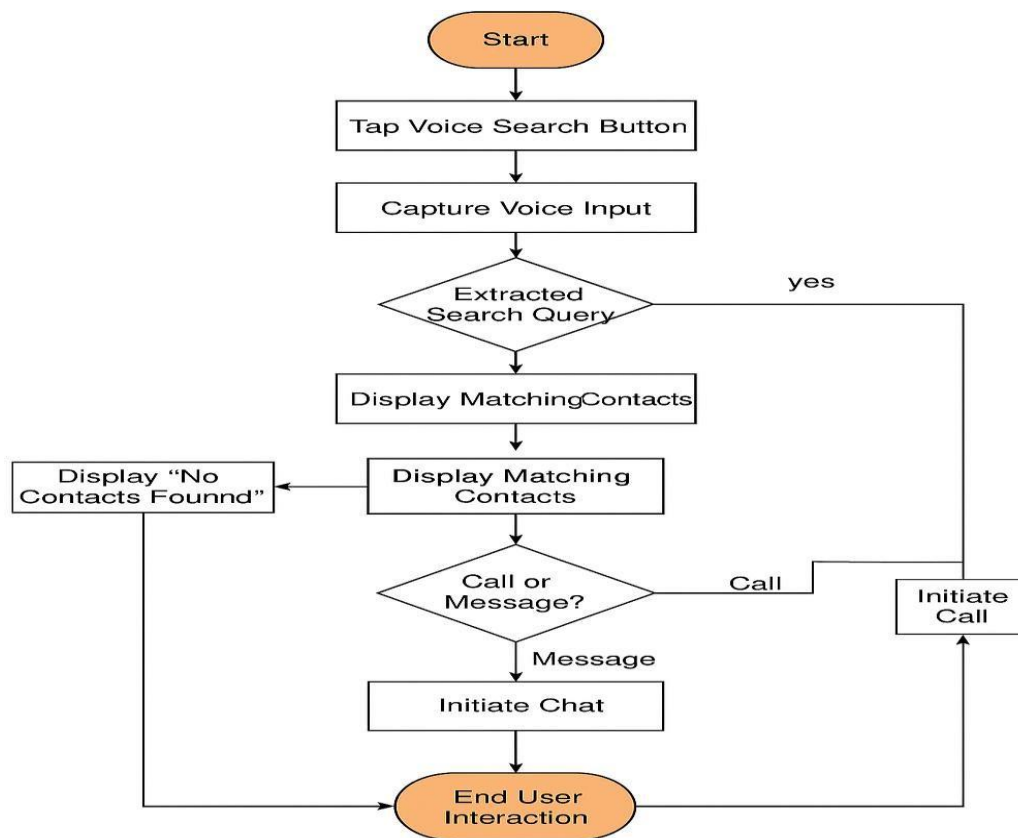


Fig 4.2.4 Activity Diagram

5.IMPLEMENTATION

5.1 TECHNOLOGIES

HTML: HTML (HyperText Markup Language) is the standard language used to create and structure content on the web. It defines the layout of a webpage using elements like headings, paragraphs, links, images, and forms. In this project, HTML is used to build the structure of the WhatsApp-like interface, including the header, search bar, microphone button, and contact list.

CSS: CSS (Cascading Style Sheets) is a stylesheet language used to describe the appearance and layout of HTML elements on a webpage. It controls aspects like colors, fonts, spacing, positioning, and responsiveness. In this project, CSS styles the WhatsApp-like interface, giving it a clean, mobile-friendly look with a green header, styled search bar, contact cards, buttons, and scroll behavior to mimic the actual WhatsApp UI.

JAVASCRIPT: JavaScript is a dynamic scripting language used to create interactive and responsive elements on web pages. In this project, JavaScript handles the voice input feature, contact search filtering, microphone interaction using the Web Speech API, and dynamic DOM manipulation to display contact details, call and message options—replicating core functionality of WhatsApp’s contact search behavior.

5.2 TOOLS

5.2.1 : Visual Studio Code: Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft. It supports multiple programming languages like HTML, CSS, and JavaScript, and includes features such as syntax highlighting, IntelliSense (smart code completion), integrated terminal, and extensions for debugging and version control. In this project, VS Code was used to write, edit, and organize the code efficiently in separate files for a structured WhatsApp-like voice search application.

5.3 MODULES

1.User Interface Module

- Responsible for displaying the WhatsApp-like layout using HTML and CSS.
- Includes the search bar, microphone button, contact list, and bottom navigation (Chats,Status,Calls).
- Provides a responsive design that adapts to various screen sizes and devices (desktop,tablet,mobile).
- Uses Bootstrap or CSS frameworks for consistent styling and layout components.
- Implements visual feedback for voice input (e.g., microphone animation while listening).
- Displays dynamic elements such as real-time contact filtering and voice-to-text output.

2 Voice Input Module

- Uses Web Speech API (specifically webkitSpeechRecognition) to capture voice input from the user.
- Converts spoken commands to text and processes the input for contact search.
- Handles various voice events such as start, end, error, and result for smooth user experience.
- Automatically populates the search bar with recognized text in real-time.
- Provides **visual indicators (e.g., animated microphone icon) during active listening.**

3. Search & Filter Module

- Uses JavaScript-based logic to filter contacts in real-time based on voice or text input.
- Dynamically supports partial keyword matching, enabling users to find contacts with incomplete names or phrases. the UI to display only the matched contacts as the user types or speaks.
- Optimized for performance with debounce techniques to reduce excessive DOM updates.
- Designed to scale for large contact lists with efficient filtering algorithms.

4. Contact Management Module

- Holds the contacts array, containing names, phone numbers, and optional profile photos.
- Manages data structure for storing and accessing contact information efficiently.
- Supports dynamic updates to the contact list based on user interactions (e.g., add/remove/edit contact).
- Fetches and stores contact data from a backend database (e.g., MongoDB) or local storage.
- Supports sorting of contacts alphabetically or by frequency of interaction (future enhancement).
- Can be extended to include additional metadata (e.g., last seen, contact type, status messages).
- Integrates with the search and filter module to present accurate results in real-time.

6.CODING

HTML:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Voice Search with Contacts</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
<div class="whatsapp-container">
<header>
<div id="whatsapp-heading">WhatsApp</div>
<div class="header-right">
<button id="moreOptionsBtn">...</button>
</div>
</header>

<div class="search-container">
<input type="text" id="searchInput" placeholder="Search contacts" />
<button id="voiceSearchBtn"> 🎤 </button>
</div>

<ul id="contacts">
<!-- Contact list will be displayed here -->
</ul>

<div id="contactOptions">
<button id="callOption" class="option-btn">Call</button>
<button id="messageOption" class="option-btn">Message</button>
</div>

<div id="actionButtons">
<button class="option-btn">Chats</button>
<button class="option-btn">Status</button>
<button class="option-btn">Calls</button>
</div>

<button id="backBtn">Back</button>

<div id="optionsMenu" style="display: none;">
<ul>
<li>New Group</li>
```



```

</li>Linked Devices</li>
</li>Payments</li>
</li>Settings</li>
</ul>
</div>
</div>

<script src="script.js"></script>
</body>
</html>

```

CSS:

```

/* Basic styling */
body {
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 0;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  background-color: hsl(240, 20%, 97%);
}

.whatsapp-container {
  width: 400px;
  background-color: rgb(249, 250, 253);
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0, 0, 0, 0.1);
  overflow: hidden;
}

header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #075e54;
  padding: 10px 20px;
  color: rgb(255, 255, 255);
}

#whatsapp-heading {
  font-size: 30px;
}

.header-right button {

```

```

    background-color: transparent;
    border: none;
    color: white;
    font-size: 24px;
    cursor: pointer;
}

.search-container {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 10px 20px;
    border-top: 1px solid #ccc;
}

#searchInput {
    width: 80%;
    padding: 5px 10px;
    border: 1px solid #ccc;
    border-radius: 20px;
}

#voiceSearchBtn {
    background-color: transparent;
    border: none;
    cursor: pointer;
    font-size: 24px;
    color: #075e54;
}

#contacts {
    list-style-type: none;
    padding: 0;
    margin: 0;
    border-top: 1px solid #ccc;
    max-height: 300px;
    overflow-y: auto;
}

li {
    display: flex;
    align-items: center;
    padding: 10px;
    border-bottom: 1px solid #eee;
}

.contact-photo {
    width: 30px;

```

```

    height: 30px;
    border-radius: 50%;
    margin-right: 10px;
}

.contact-info {
    flex: 1;
}

.contact-name {
    font-weight: bold;
}

#contactOptions {
    display: none;
    padding: 10px;
    text-align: center;
}

#actionButtons {
    display: flex;
    justify-content: space-around;
    padding: 10px 0;
    border-top: 1px solid #ccc;
    background-color: #f5f5f5;
}

#backBtn {
    display: none;
    margin-top: 10px;
    padding: 5px 15px;
    background-color: #075e54;
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 5px;
}

.option-btn {
    padding: 5px 15px;
    background-color: #075e54;
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 5px;
}

```

JAVASCRIPT:

```
// Example contacts array with name, phone number, and profile photo
const contacts = [
  { name: 'John Doe', phone: '9876543210', photo: 'https://via.placeholder.com/30' },
  { name: 'Amit Kumar', phone: '9887654321', photo: 'https://via.placeholder.com/30' },
  { name: 'Sanjay Gupta', phone: '9998765432', photo: 'https://via.placeholder.com/30' },
  { name: 'Priya Sharma', phone: '9456789101', photo: 'https://via.placeholder.com/30' },
  { name: 'Ravi Patel', phone: '9123456789', photo: 'https://via.placeholder.com/30' },
  { name: 'Anjali Verma', phone: '9223344556', photo: 'https://via.placeholder.com/30' },
  { name: 'Vikram Reddy', phone: '9311122334', photo: 'https://via.placeholder.com/30' },
  { name: 'Meena Iyer', phone: '9322114455', photo: 'https://via.placeholder.com/30' },
  { name: 'Rajesh Singh', phone: '9333223344', photo: 'https://via.placeholder.com/30' },
  { name: 'Neha Joshi', phone: '9444335566', photo: 'https://via.placeholder.com/30' },
];

// Elements
const searchInput = document.getElementById('searchInput');
const contactsListContainer = document.getElementById('contacts');
const backBtn = document.getElementById('backBtn');
const contactOptions = document.getElementById('contactOptions');
const callOption = document.getElementById('callOption');
const messageOption = document.getElementById('messageOption');
const optionsMenu = document.getElementById('optionsMenu');
const moreOptionsBtn = document.getElementById('moreOptionsBtn');
const voiceSearchBtn = document.getElementById('voiceSearchBtn');
const actionButtons = document.getElementById('actionButtons');

// Initialize contacts in the list
function displayContacts(contacts) {
  contactsListContainer.innerHTML = ""; // Clear the current contact list
  contacts.forEach(contact => {
    const listItem = document.createElement('li');
    listItem.innerHTML = `
      
      <div class="contact-info">
        <div class="contact-name">${contact.name}</div>
        <div>${contact.phone}</div>
      </div>`;
    listItem.addEventListener('click', () => openContact(contact));
    contactsListContainer.appendChild(listItem);
  });
}

// Filter contacts based on search input
function filterContactsBySearch(query) {
  const searchQuery = query.toLowerCase();
```

```

const contactItems = document.querySelectorAll('#contacts li');
contactItems.forEach(item => {
  const name = item.querySelector('.contact-name').textContent.toLowerCase();
  if (name.includes(searchQuery)) {
    item.style.display = 'flex'; // Show contact
  } else {
    item.style.display = 'none'; // Hide contact
  }
});
}

// Show contact options (Call, Message) when a contact is clicked
function openContact(contact) {
  contactOptions.style.display = 'block'; // Show contact options
  callOption.textContent = Call ${contact.name};
  messageOption.textContent = Message ${contact.name};
  actionButtons.style.display = 'none'; // Hide action buttons
  backBtn.style.display = 'inline-block'; // Show back button
}

// Back to home page
backBtn.addEventListener('click', () => {
  actionButtons.style.display = 'flex'; // Show action buttons again
  backBtn.style.display = 'none'; // Hide back button
  contactOptions.style.display = 'none'; // Hide contact options
  searchInput.value = ''; // Clear the search input
  displayContacts(contacts); // Display the full contacts list
});

// Voice Recognition Functionality
if ('webkitSpeechRecognition' in window) {
  const recognition = new webkitSpeechRecognition();
  recognition.continuous = false;
  recognition.lang = 'en-US'; // Set language to English
  recognition.interimResults = false;

  // Start recognition when mic button is clicked
  voiceSearchBtn.addEventListener('click', () => {
    recognition.start();
  });

  // Capture the speech input and fill the search input field
  recognition.onresult = function(event) {
    const transcript = event.results[0][0].transcript;
    searchInput.value = transcript; // Place the spoken text in the input field
    filterContactsBySearch(transcript); // Filter contacts based on the search
  };
}

```

```

// Handle recognition errors
recognition.onerror = function(event) {
    console.error('Speech recognition error:', event.error);
};

// Handle end of recognition
recognition.onend = function() {
    console.log('Speech recognition ended.');
```

```

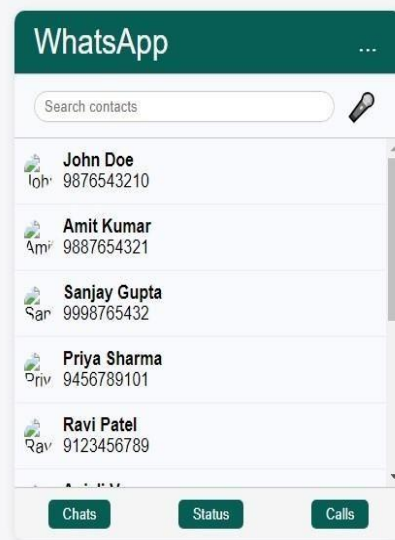
};
}

// Display all contacts initially
displayContacts(contacts);

// More options button
moreOptionsBtn.addEventListener('click', () => {
    optionsMenu.style.display = optionsMenu.style.display === 'none' ? 'block' : 'none';
});

```

7.OUTPUT SCREENS:





8.CONCLUSION:

The “Smart Voice Search for Contact Management” system presents a modern and efficient approach to managing digital contacts by integrating voice recognition capabilities within a web-based application. Unlike traditional systems that depend heavily on manual text input, this application leverages the **Web Speech API** to enable hands-free contact search, thereby improving usability, accessibility, and speed.

Its WhatsApp-like user interface ensures familiarity for users, while additional functionalities such as group creation, device linking, and customizable settings enrich the user experience. The modular structure of the system—including user interface, contact management, search filtering, voice recognition, and navigation—promotes clean architecture and future scalability.

By allowing users to interact through both voice and text, the system caters to a broader audience, including users with disabilities and those in multitasking scenarios. The implementation of **HTML, CSS, JavaScript, Node.js, and MongoDB** ensures that the system is not only interactive and responsive but also maintainable and scalable for larger applications. The use of `webkitSpeechRecognition` for speech-to-text conversion ensures fast and accurate voice input, enhancing the overall efficiency of the system.

This project successfully demonstrates how voice technology can revolutionize user interaction in web applications, offering a smarter and more accessible alternative to conventional contact search methods.

It lays the groundwork for further development, such as integrating artificial intelligence for predictive contact suggestions or incorporating multilingual voice recognition. Ultimately, this system bridges the gap between functionality and user convenience, creating a more inclusive and advanced contact management solution for the digital age.

Future Enhancements

- Multilingual voice recognition support.
- AI-based contact prediction or auto-suggestions.
- Real-time synchronization with mobile contacts via backend (e.g., Firebase or MongoDB).
- Voice command support for other actions (e.g., “Call last contact,” “Show missed calls”).
- Integration with authentication and security layers for real-world deployment.
- Dark mode and customizable themes for improved user experience and accessibility.
- Offline voice search support using local storage and caching for limited connectivity
- Integration with calendar and reminders, allowing users to set events or alerts via voice
- Contact grouping and smart categorization using machine learning for better organization.
- Support for voice feedback/output, where the system can read contact names or confirmations aloud.

9. REFERENCES:

Frontend Development:

1. Web Speech API Documentation
 1. Guide on implementing speech recognition for voice search.
 2. URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
2. HTML, CSS, and JavaScript Basics
 1. Resources for building a responsive and interactive user interface.
 2. URL: <https://www.w3schools.com>
3. Bootstrap or CSS Frameworks for UI Design
 1. CSS libraries to enhance the look and feel of the application.
 2. URL: <https://getbootstrap.com>

Backend Development:

4. **Node.js and Express.js Documentation**
 4. Guide for setting up a backend server and handling API requests.
 5. URL: <https://expressjs.com/>
5. **MongoDB Documentation (Database)**
 4. Guide on creating and managing a NoSQL database to store contact information.
 5. URL: <https://www.mongodb.com/docs/>
6. **Mongoose Documentation**
 4. ORM to facilitate communication between MongoDB and Node.js.
 5. URL: <https://mongoosejs.com/>
7. **REST API Design Guidelines**
 4. Best practices for designing and securing REST APIs for contact management.
 5. URL: <https://www.restapitutorial.com>